SC2006 Lab 4 - AI Coding Agent Reflections

Project: Smart Parking Recommendation App, Team 1 | **Scope:** Lab 4 Section 3.3, coding-agent exercises

Experiment 1: Reconstructing the recommendation control

We asked the coding agent to regenerate the implementation corresponding to the BCE RecommendationEngine control. In our TypeScript project, the selected control operation is `calculateRecommendationScore()`, a function rather than a class. The agent was given SRS FR44-FR49, the architecture description, shared types, the black-box tests from Section 3.2, and the surrounding project. The implementation file was removed only in an isolated worktree. Before removal, all ten scorer tests passed; after removal the test file could not load. The agent then wrote a new implementation. Its first attempt compiled and passed all ten scorer tests, and the full backend suite passed 300 tests. No correction prompt was needed.

The outcome met the functional expectations checked by our tests. The generated version used a single ETA-weight calculation and explicit branches, while the original used a different structure. This result has an important limitation: the agent had already seen the original implementation earlier in the lab work. We therefore cannot treat the result as a blind reconstruction from requirements. A future run with a fresh agent that has never seen the original would make the comparison stronger. Human review is still needed for behaviour outside the tested input classes and for constants the SRS does not specify.

Experiment 2: Correcting the historical-arrival requirement mismatch

For the selected requirements-mismatch experiment, we asked the agent to check whether historical availability used the carpark's estimated arrival day and hour, as FR45 requires. We provided the SRS, search orchestration and routing code, the historical lookup and tests, and the Section 3.2 reports. The agent found that the search service used the search-time day and hour for every carpark, before the individual driving ETAs were calculated.

The agent first added seven regression cases in a separate worktree. Before the correction, one same-hour case passed and six cases failed, including next-hour, midnight, weekday rollover, different carpark ETAs, and missing ETA. The agent then changed the lookup to use each carpark's driving ETA and Singapore local time. When driving ETA is unavailable, it skips historical lookup and uses the existing real-time fallback; this is an assumption for team review because the SRS does not define a precise fallback bucket. After this single correction, all seven new tests and the full 307-test backend suite passed. The 153 frontend tests, type checks, and lint also passed. No additional correction iteration was needed.

This result met our tested requirement more convincingly than Experiment 1 because the regression tests failed on the original code and passed after the correction. The tests still mock external APIs and the database, so the team should review the fallback decision and test the behaviour with real historical records before merging. The correction remains isolated; it has not replaced the main implementation.

Supporting results

Check	Experiment 1	Experiment 2
New targeted cases	10/10 scorer tests passed	7/7 arrival tests passed after 1/7 before
Full backend	300/300 passed	307/307 passed
Frontend	Not rerun for the scorer-only experiment	153/153 passed
Type checking	Backend passed	Backend, frontend, shared types passed
Lint	Not rerun for the scorer-only experiment	Passed

Full prompts, procedure, file paths, commands, and limitations are recorded in `testing/ai_experiments/experiment_1.md` and `experiment_2.md`.